

Traivo - Prestandaoptimering för Snabb & Responsiv Upplevelse

Executive Summary

Omfattande optimeringsförslag för att göra Traivo-plattformen blixtsnabb och eliminera väntetider. Fokus på upplevd prestanda, faktisk prestanda och skalbarhet för flerföretagsmiljö.

1. Frontend-Optimering - Upplevd Prestanda

1.1 Initial Load Optimization

Problem: Långsam första laddning skapar dåligt första intryck.

Lösningar:

- **Code Splitting:** Dela upp JavaScript i chunks, ladda endast vad som behövs för varje vy
- Använd dynamic imports: `const Scheduler = lazy(() => import('./Scheduler'))`
- Route-based splitting för varje huvudsektion
- Vendor bundle separation

- **Progressive Web App (PWA):**

- Service Worker för instant återbesök
- Offline-first arkitektur för fältarbetare
- App Shell Model - ladda UI-skal först, innehåll sedan

- **Critical CSS Inlining:**

- Inline above-the-fold CSS i `<head>`
- Lazy load resten av stilarna
- Eliminating render-blocking resources

- **Resource Hints:**

```
html
<link rel="preconnect" href="https://api.traivo.com">
<link rel="dns-prefetch" href="https://api.traivo.com">
<link rel="preload" href="/fonts/main.woff2" as="font">
```

Förväntad förbättring: 60-70% snabbare First Contentful Paint (FCP)

1.2 Optimistic UI Updates

Problem: Användare väntar på server-bekräftelse för varje action.

Lösningar:

- **Immediate Feedback:** Uppdatera UI direkt, synka med backend i bakgrunden
``javascript

```
// Exempel: Tilldela tekniker till jobb
function assignTechnician(jobId, technicianId) {
// 1. Uppdatera UI OMEDELBART
updateUIOptimistically(jobId, technicianId);
```

```
// 2. Synka med server i bakgrund
api.assignJob(jobId, technicianId)
  .catch(error => {
    // 3. Rollback vid fel
    revertUIChange(jobId);
    showErrorToast(error);
  });
```

```
}
...
```

- **Skeleton Screens:** Visa innehållsplaceholders istället för spinners
- **Progressive Enhancement:** Visa data inkrementellt när den laddas

Förväntad förbättring: Upplevs 3-5x snabbare för användarinteraktioner

1.3 Intelligent Prefetching

Problem: Varje sidövergång kräver ny data-fetch.

Lösningar:

- **Route Prefetching:** Förutse nästa sida baserat på användarens beteende
- Hover över "Schema"-knappen → preload schema-data
- Användare öppnar ofta jobb #123 efter jobb #122 → preload nästa jobb
 - **Time-based Prefetching:**
 - Ladda dagens schema kl 06:00 automatiskt
 - Preload vanliga rapporter innan EOD
 - **ML-driven Prefetching:** AI förutser användarens nästa steg baserat på historik

Förväntad förbättring: Upplevs som instant navigation (0-100ms istället för 500-1500ms)

1.4 Virtual Scrolling & Pagination

Problem: Listor med 1000+ jobb/tekniker krashar eller laggar.

Lösningar:

- **Virtual Scrolling:** Rendera endast synliga rader (React Window / TanStack Virtual)
- 10,000 jobb → endast ~20 DOM-noder renderade
 - **Infinite Scroll med Windowing:** Ladda mer data när användaren scrollar
 - **Smart Pagination:**
 - Cursor-based pagination (snabbare än offset)
 - Cache föregående sidor

Förväntad förbättring: Hantera 10x mer data utan performance-degradation

2. Backend-Optimering - API Prestanda

2.1 GraphQL eller tRPC istället för REST

Problem: Over-fetching/under-fetching med REST API, många roundtrips.

Lösningar:

- **GraphQL:** Låt frontend hämta exakt vad den behöver i en query

```
graphql
query GetDashboard {
  jobs(today: true) { id, status, technician { name } }
  technicians(available: true) { id, name, location }
  alerts { id, message }
}
```

- **DataLoader Pattern:** Batching + caching för N+1 queries
- **tRPC alternativ:** Type-safe API med automatic serialization

Förväntad förbättring: 50-70% färre API-anrop, 40% mindre data-överföring

2.2 Response Caching Strategi

Problem: Samma data hämtas om och om igen.

Lösningar:

Nivå 1 - HTTP Caching:

```
Cache-Control: public, max-age=300, stale-while-revalidate=60
ETag: "v1.2.3"
```

Nivå 2 - Redis Application Cache:

- **Hot Data:** Dagens scheman (TTL: 5min)
- **Warm Data:** Tekniker-listor (TTL: 30min)
- **Cold Data:** Historiska rapporter (TTL: 24h)

```
# Exempel cache-strategi
@cache(ttl=300, key="schedule:{customer_id}:{date}")
def get_daily_schedule(customer_id, date):
    return db.query(...)
```

Nivå 3 - CDN för Statisk Data:

- Organisationsinfo, inställningar, konfigurationer

Cache Invalidation:

- Event-driven: När schemaändring sker → invalidera relaterad cache
- Använd Cache-Tags för granulär invalidering

Förväntad förbättring: 80-95% snabbare för cachade endpoints

2.3 Database Query Optimization

Problem: Långsamma databasfrågor blockerar requests.

Lösningar:

Indexering:

```
-- Fältservice-specifika index
CREATE INDEX idx_jobs_customer_date ON jobs(customer_id, scheduled_date);
CREATE INDEX idx_jobs_status_tech ON jobs(status, technician_id) WHERE status != 'completed';
CREATE INDEX idx_technicians_location ON technicians USING GIST(location);
```

Query Optimization:

- Använd EXPLAIN ANALYZE för alla viktiga queries
- Eager loading av relationer (förhindra N+1)
- Materialized views för komplexa rapporter
- Partition stora tabeller (t.ex. jobs per månad)

Connection Pooling:

```
# PgBouncer eller SQLAlchemy pool
engine = create_engine(
    'postgresql://...',
    pool_size=20,
    max_overflow=40,
    pool_pre_ping=True # Hantera dead connections
)
```

Read Replicas:

- Write → Primary DB
- Read (reports, listor) → Read Replicas
- Load balancing mellan replicas

Förväntad förbättring: 3-10x snabbare queries, 50% lägre DB-load

2.4 Async Processing för Tunga Operationer

Problem: AI-planering/omplanering kan ta 5-30 sekunder, blockerar användaren.

Lösningar:

Background Jobs med Celery/Redis:

```
# Användaren klickar "Optimera schema"
@app.post("/optimize-schedule")
def optimize_schedule(date, customer_id):
    # 1. Skapa jobb direkt (5ms)
    task = optimize_task.delay(customer_id, date)

    # 2. Returnera task_id omedelbart
    return {"task_id": task.id, "status": "processing"}

# Tungt jobb körs i bakgrunden
@celery_app.task
def optimize_task(customer_id, date):
    result = ai_optimizer.optimize(customer_id, date)
    # 3. Notifiera frontend när klar (WebSocket)
    notify_user(customer_id, result)
```

Real-time Updates via WebSocket:

```
// Frontend lyssnar
socket.on('schedule_optimized', (data) => {
    updateScheduleView(data);
    showNotification('Schema optimerat!');
});
```

Progress Indicators:

- "Optimerar 47 jobb... 68% klart"
- Visa delresultat inkrementellt

Förväntad förbättring: Användaren kan arbeta vidare direkt, upplevs som instant

2.5 API Rate Limiting & Throttling

Problem: Buggy frontend gör 1000 requests/sekund, krashar systemet.

Lösningar:

- **Per-User Rate Limiting:** 100 req/min per användare
- **Per-Customer Rate Limiting:** 1000 req/min per företag
- **Intelligent Throttling:** Dynamisk baserad på server-load
- **Retry med Exponential Backoff:** Client väntar 1s, 2s, 4s...

Förväntad förbättring: Stabilitet, förhindrar cascade failures

3. AI/ML Optimering - Snabba Planeringsbeslut

3.1 Hybrid AI Approach

Problem: Full AI-optimering tar för lång tid för real-time behov.

Lösningar:

Instant Heuristics (< 100ms):

- Regelbaserad allokering för enkla fall

- Snabb approximation baserad på geografisk närhet + tillgänglighet
- Används för drag-and-drop i UI

Smart AI (1-5 sekunder):

- ML-modell för mer komplexa optimeringsproblem
- Körs i bakgrunden, föreslår förbättringar

Deep Optimization (30+ sekunder):

- Full constraint-solving för nattliga re-optimeringskörningar
- Batch-optimering av hela veckan

```
def assign_job(job, technicians):
    # Quick heuristic for UI feedback
    quick_match = heuristic_matcher(job, technicians) # 50ms
    update_ui_optimistically(quick_match)

    # AI refinement i bakgrund
    ai_match = ml_optimizer.optimize_async(job, technicians) # 2s
    if ai_match.score > quick_match.score:
        suggest_better_match(ai_match)
```

Förväntad förbättring: Instant feedback + smart optimization

3.2 Model Caching & Pre-computation

Problem: AI-modeller laddas om för varje request.

Lösningar:

- **Model Serving:** TensorFlow Serving, TorchServe, eller Triton
- **Warm Models:** Håll modeller laddade i minnet
- **Pre-computed Features:** Beräkna tekniker-kompetenser, reseavstånd i förväg
- **Embedding Cache:** Spara vektorrepresentationer för jobb/tekniker

Förväntad förbättring: 5-10x snabbare inference

3.3 Incremental Updates istället för Full Recalculation

Problem: Ett nytt jobb triggar re-optimering av 1000 existerande jobb.

Lösningar:

- **Delta Updates:** Optimera endast påverkade jobb/tekniker
- **Local Search:** Förbättra lösning inkrementellt
- **Conflict Detection:** Identifiera endast de som måste flyttas

Förväntad förbättring: 10-100x snabbare för små ändringar

4. Multi-Tenant Optimization

4.1 Data Isolation Strategy

Problem: En kunds tunga operation påverkar alla andra.

Lösningar:

Database-nivå:

```
-- Row-Level Security (RLS)
CREATE POLICY customer_isolation ON jobs
  USING (customer_id = current_setting('app.customer_id')::uuid);

-- eller Separate Schemas per kund
CREATE SCHEMA customer_123;
```

Application-nivå:

- Connection pooling per tenant
- Resource quotas per kund
- Priority queueing för enterprise-kunder

Caching-nivå:

- Separate cache namespaces: `cache:customer_{id}:*`
- Prevent cache poisoning mellan tenants

Förväntad förbättring: Isolerad prestanda, en kunds burst påverkar inte andra

4.2 Smart Resource Allocation

Problem: Små kunder får samma resurser som stora.

Lösningar:

- **Tiered Service:** Enterprise får dedicerade workers
- **Adaptive Throttling:** Större buffertar för premium-kunder
- **Prioriterad Queueing:** VIP-kunder först i kön för AI-optimering

Förväntad förbättring: Bättre resursutnyttjande, nöjdare betalande kunder

5. Real-Time Features utan Overhead

5.1 Efficient WebSocket Architecture

Problem: Polling var 5:e sekund skapar onödig belastning.

Lösningar:

WebSocket för Kritiska Updates:

```
// Endast viktiga events
socket.on('job_assigned', updateScheduler);
socket.on('technician_location', updateMap);
socket.on('job_completed', refreshStats);
```

Selective Broadcasting:

- Skicka endast till påverkade användare
- Room-based subscriptions: `socket.join(customer_${id})`

Connection Pooling:

- WebSocket gateway (Soketi, Socket.io cluster)
- Redis Pub/Sub för horizontal scaling

Fallback Strategy:

- WebSocket först
- Long-polling om WebSocket misslyckas
- Graceful degradation

Förväntad förbättring: 95% mindre API-trafik, instant updates

5.2 Smart Data Synchronization

Problem: Onödiga uppdateringar när data inte ändrats.

Lösningar:

- **Event Sourcing:** Skicka endast ändringar (deltas)

json

```
{ "event": "job_updated", "job_id": 123, "changes": { "status": "in_progress" } }
```

- **Version Tracking:** ETag/version för konfliktshantering
- **Operational Transformation:** För collaborative editing

Förväntad förbättring: 70% mindre data-överföring

6. Mobile & Field Worker Optimization

6.1 Offline-First Architecture

Problem: Fältarbetare har spotty internet, kan inte uppdatera jobb.

Lösningar:

Service Worker + IndexedDB:


```
// Cache kritisk data lokalt
const cache = await caches.open('traivo-v1');
await cache.addAll([
  '/today-schedule',
  '/technician-profile',
  '/offline-form'
]);

// Sync när online igen
self.addEventListener('sync', event => {
  if (event.tag === 'sync-jobs') {
    event.waitUntil(syncPendingJobs());
  }
});
```

Background Sync API:

- Jobbuppdateringar synkas automatiskt när anslutning återställs
- Queue-baserad sync

Conflict Resolution:

- Last-write-wins för enkla fält
- Operational transforms för komplex data
- Manual resolution för kritiska konflikter

Förväntad förbättring: Fungerar utan internet, 0 data loss

6.2 Mobile-Specific Optimizations

Problem: Mobil-app känns trögare än desktop.

Lösningar:

- **Adaptive Loading:** Ladda low-res bilder på dålig connection
- **Request Prioritization:** GPS-uppdateringar framför analytics
- **Battery-aware:** Minska sync-frekvens vid lågt batteri
- **Native Mobile App:** React Native/Flutter för bättre prestanda än webb

Förväntad förbättring: 2-3x snabbare mobile experience

7. Monitoring & Performance Budgets

7.1 Real User Monitoring (RUM)

Implementera:

- **Core Web Vitals tracking:**
 - LCP (Largest Contentful Paint): < 2.5s
 - FID (First Input Delay): < 100ms
 - CLS (Cumulative Layout Shift): < 0.1
- **Custom Metrics:**
 - Time to Interactive för scheduler-view
 - API response time per endpoint

- AI optimization duration

Tools: Sentry, DataDog RUM, New Relic

7.2 Performance Budgets

Sätt hårda gränser:

```
budgets:  
  - resourceType: script  
    budget: 300kb  
  - resourceType: total  
    budget: 1000kb  
  - metric: interactive  
    budget: 3s
```

CI/CD Integration: Fail build om budget överskrids

Förväntad förbättring: Förhindrar performance regression

8. Infrastructure Optimization

8.1 Auto-Scaling & Load Balancing

Problem: Morgonrusning kl 08:00 krashar systemet.

Lösningar:

- **Horizontal Pod Autoscaling (Kubernetes):**

```
yaml  
apiVersion: autoscaling/v2  
kind: HorizontalPodAutoscaler  
spec:  
  minReplicas: 3  
  maxReplicas: 20  
  metrics:  
    - type: Resource  
      resource:  
        name: cpu  
        target:  
          type: Utilization  
          averageUtilization: 70
```

- **Predictive Scaling:** Skala upp proaktivt kl 07:45 baserat på historik
- **Geographic Load Balancing:** Route till närmaste datacenter

Förväntad förbättring: Klarar 10x traffic spike utan slowdown

8.2 Edge Computing & CDN

Problem: Geografiskt spridda användare upplever latency.

Lösningar:

- **Edge Functions:** Cloudflare Workers, AWS Lambda@Edge
- Kör enkel logik nära användaren
- Personalisering utan roundtrip till origin
 - **Static Asset CDN:** CloudFront, Cloudflare
 - Bilder, CSS, JS serveras från edge
 - **API Edge Caching:** Cache vissa endpoints på CDN (med tenant-aware keys)

Förväntad förbättring: 50-200ms lägre latency globalt

9. Kodoptimering - Best Practices

9.1 Frontend Performance Patterns

React/Vue Optimization:

```
// Memoization för dyra beräkningar
const expensiveValue = useMemo(() =>
  calculateComplexSchedule(jobs),
  [jobs]
);

// Callback memoization
const handleJobClick = useCallback((jobId) => {
  openJobDetails(jobId);
}, []);

// Virtual lists för stora dataset
import { FixedSizeList } from 'react-window';

// Lazy loading av komponenter
const JobDetails = lazy(() => import('./JobDetails'));
```

Debounce & Throttle:

```
// Sök-input: debounce
const searchJobs = debounce(query => api.search(query), 300);

// Scroll events: throttle
const handleScroll = throttle(event => updateMap(), 100);
```

9.2 Backend Performance Patterns

N+1 Query Prevention:

```
# Dåligt (N+1)
jobs = Job.query.all()
for job in jobs:
    print(job.technician.name) # Ny query per iteration

# Bra (Eager loading)
jobs = Job.query.options(joinedload(Job.technician)).all()
for job in jobs:
    print(job.technician.name) # Ingen extra query
```

Bulk Operations:

```
# Dåligt
for job in jobs:
    db.session.add(job)
    db.session.commit() # 1000 commits

# Bra
db.session.bulk_insert_mappings(Job, jobs)
db.session.commit() # 1 commit
```

10. Quick Wins - Implementera Första Veckan

Prioriterad checklista:

Dag 1-2: Frontend

- [] Implementera skeleton screens
- [] Lägg till loading states och optimistic UI
- [] Enable gzip/brotli compression
- [] Minify och bundle JavaScript

Dag 3-4: Backend

- [] Lägg till Redis för hot-data caching
- [] Implementera database indexing
- [] Enable HTTP caching headers
- [] Connection pooling

Dag 5: Monitoring

- [] Setup Sentry eller liknande för error tracking
- [] Implementera basic performance metrics
- [] Alert för slow queries (>1s)

Förväntad förbättring från Quick Wins: 2-3x snabbare system

11. Långsiktiga Optimeringar (Månad 2-3)

Arkitektur:

- [] Migrera till GraphQL API
- [] Implementera event-driven arkitektur

- [] Microservices för isolerad skalning
- [] Edge computing för global latency

AI/ML:

- [] Model serving infrastructure
- [] A/B testing av optimeringsalgoritmer
- [] Reinforcement learning för kontinuerlig förbättring

Infrastructure:

- [] Multi-region deployment
- [] Disaster recovery & failover
- [] Chaos engineering för resilience testing

12. KPI:er för Att Mäta Framgång

User Experience Metrics

- **Time to Interactive:** < 3 sekunder
- **API Response Time p95:** < 500ms
- **Page Load Time:** < 2 sekunder
- **Mobile Performance Score:** > 90/100

Business Metrics

- **User Abandonment Rate:** < 5%
- **Daily Active Users:** ökar med bättre prestanda
- **Feature Adoption:** snabbare features = högre adoption
- **Support Tickets (performance):** reducera med 70%

Technical Metrics

- **Cache Hit Ratio:** > 80%
- **Database Query Time p95:** < 100ms
- **Error Rate:** < 0.1%
- **Uptime:** 99.9%

13. Implementationsplan

Fas 1: Foundation (Vecka 1-2)

Fokus: Quick wins, monitoring, grundläggande caching

Team: 2 frontend + 2 backend devs

Kostnad: Låg

Impact: 2-3x förbättring

Fas 2: Core Optimization (Vecka 3-6)

Fokus: Database, API, AI-optimering

Team: Full team + DevOps

Kostnad: Medel

Impact: 5-10x förbättring

Fas 3: Advanced Features (Månad 2-3)

Fokus: Real-time, offline, edge computing

Team: Full team + specialist

Kostnad: Hög

Impact: Professionell-grade platform

Fas 4: Continuous Improvement (Ongoing)

Fokus: A/B testing, monitoring, iterativ förbättring

Team: Dedikerad performance team

Kostnad: Ongoing

Impact: Behåll konkurrensfördel

14. Budget & ROI-Estimat

Infrastructure Costs

- **Redis Cluster:** €200-500/månad
- **CDN (Cloudflare/CloudFront):** €100-300/månad
- **Monitoring (DataDog/New Relic):** €200-500/månad
- **Additional Compute:** €500-1500/månad (auto-scaling)

Total: ~€1,000-3,000/månad

ROI

- **Ökad User Retention:** +15-25% (snabbare = nöjdare kunder)
- **Högre Conversion:** +10-20% (mindre abandoned sessions)
- **Reduced Support Costs:** -30% (färre performance-relaterade tickets)
- **Competitive Advantage:** Omätbar men kritisk

Payback Period: 2-4 månader för enterprise-kunder

15. Risk & Mitigation

Risk	Sannolikhet	Impact	Mitigation
Over-engineering	Medel	Medel	Börja med quick wins, iterera
Breaking changes	Låg	Hög	Feature flags, gradual rollout
Cache inconsistency	Medel	Medel	Robust invalidation, monitoring
Cost overrun	Medel	Låg	Budget alerts, gradual scaling
Team bandwidth	Hög	Medel	Prioritera, external konsulter

Sammanfattning - Top 5 Rekommendationer

1. Optimistic UI + Async Processing

Varför: Största impact på upplevd prestanda

Effort: Medel

Impact: Upplevs 3-5x snabbare

2. Redis Caching Strategi

Varför: Dramatiskt reducerar DB-load och response time

Effort: Låg-medel

Impact: 80-95% snabbare för hot data

3. Database Indexering + Query Optimization

Varför: Flaskhals i de flesta system

Effort: Medel

Impact: 3-10x snabbare queries

4. Code Splitting + Lazy Loading

Varför: Snabbare initial load, bättre mobile experience

Effort: Låg-medel

Impact: 60-70% snabbare FCP

5. WebSocket för Real-Time Updates

Varför: Eliminera polling, instant feedback

Effort: Medel

Impact: 95% mindre trafik, instant updates

Nästa Steg

1. **Review denna plan** med tech team och prioritera baserat på nuvarande arkitektur
 2. **Performance Audit:** Kör Lighthouse, WebPageTest för baseline metrics
 3. **Identifiera Top 3 Bottlenecks:** Profiling av backend + frontend
 4. **Pilotprojekt:** Välj 1-2 optimeringar för proof-of-concept
 5. **Iterera:** Mät, optimera, repetera
-

Dokument skapat: 2026-05-06

Författare: Abacus AI Agent

Version: 1.0